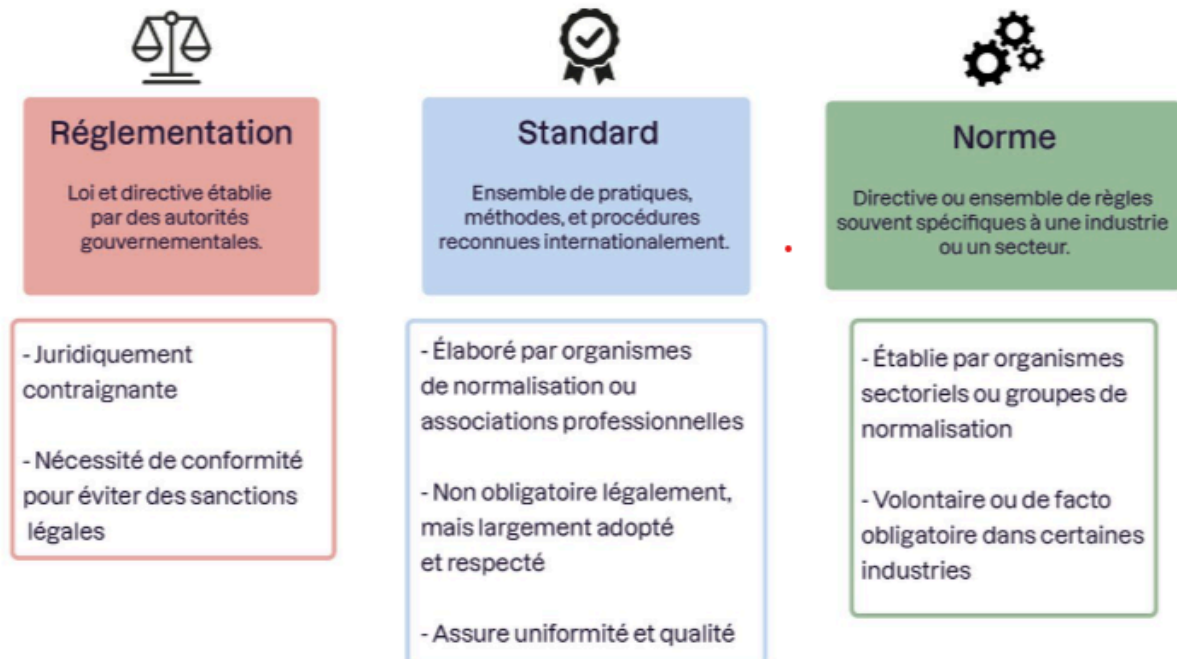


Différences entre Réglementation, Norme et Standard



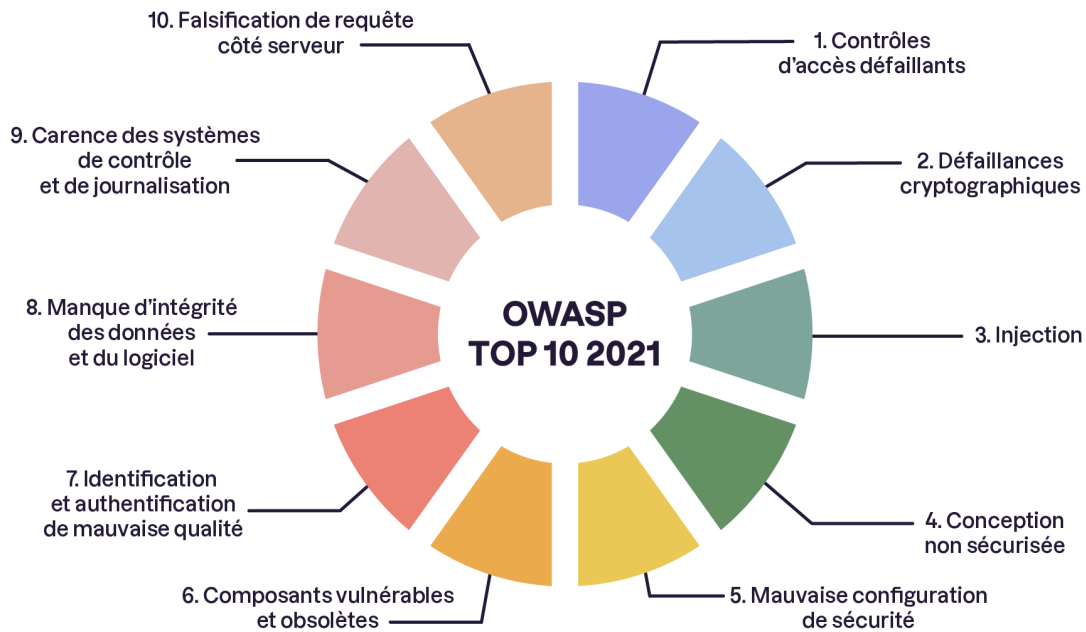
La norme ISO/IEC 27001

La norme ISO/IEC 27001 est une norme internationale qui encourage une approche basée sur le risque pour la gestion de la sécurité de l'information.

Le standard PCI-DSS

Le standard **PCI-DSS** (Payment Card Industry Data Security Standard) est un excellent exemple de standard défini pour sécuriser les transactions par carte bancaire. Il impose des exigences rigoureuses pour la protection des données de carte de crédit, incluant le chiffrement des données, des mesures de contrôle d'accès, et une surveillance continue.

Les 10 familles de vulnérabilité référencées par l'OWASP en 2021



A01 : Contrôles d'accès défaillants

Le contrôle d'accès applique une stratégie telle que les utilisateurs ne peuvent pas agir en dehors de leurs autorisations prévues.

Exemples :

- L'attaque par découverte d'URL ;
- Les références directes d'objets non sécurisés (ou IDOR, pour Insecure Direct Object Reference) ;
- L'attaque CSRF (Cross-Site Request Forgery).

[Web Service Security - OWASP Cheat Sheet Series](#)

[Insecure Direct Object Reference Prevention - OWASP Cheat Sheet Series](#)

[Cross-Site Request Forgery Prevention - OWASP Cheat Sheet Series](#)

[Authorization - OWASP Cheat Sheet Series](#)

A02 : Défaillances cryptographiques

Se concentre sur les vulnérabilités d'une mise en œuvre inadéquate ou d'une utilisation incorrecte des techniques cryptographiques.

La mise en œuvre du salage varie selon le langage de programmation utilisé, mais l'idée reste la même : combiner le mot de passe avec une **donnée aléatoire** avant de le hacher. Il est important de choisir un **algorithme de hachage robuste**. Des algorithmes comme Bcrypt, Scrypt, Argon2, ou PBKDF2 sont recommandés pour leur **sécurité accrue**.

L'OWASP recommande notamment l'utilisation d'**Argon2**, tandis que l'Anssi suggère les algorithmes **SHA-2** ou **SHA-3** pour vérifier l'intégrité des fichiers et **PBKDF2** pour le hachage des mots de passe.

[Password Storage - OWASP Cheat Sheet Series](#)

[A03 Injection - OWASP Top 10:2021](#)

La catégorie **A03:2021** de l'**OWASP** fait référence à l'une des failles de sécurité les plus critiques et omniprésentes : **les injections**. Ces vulnérabilités surviennent lorsque des données non fiables sont envoyées à un interpréteur comme partie d'une commande ou d'une requête

[SQL Injection Prevention - OWASP Cheat Sheet Series](#)

[Cross Site Scripting Prevention - OWASP Cheat Sheet Series](#)

Parmi les recommandations, voici les deux principales pour cette catégorie :
Validation stricte des entrées utilisateurs : Assurez-vous que toutes les entrées fournies par les utilisateurs sont validées en fonction de critères stricts. Cela signifie refuser toutes les données qui ne correspondent pas exactement au format attendu. Utilisez des expressions régulières et des listes blanches pour valider les formats de données acceptables et rejeter toute entrée suspecte.

Échappement des caractères spécifiques : Lorsque l'utilisation de requêtes paramétrées n'est pas possible, assurez-vous d'échapper systématiquement les caractères spéciaux dans les entrées utilisateurs qui sont ensuite utilisées dans des requêtes ou des commandes.

Le principal risque lorsqu'un attaquant parvient à exploiter une **injection XSS** est le **vol de cookie**. L'un des moyens les plus efficaces pour limiter le risque de vol de cookies est d'utiliser le **flag** `HttpOnly` pour les cookies.

[A04 Insecure Design - OWASP Top 10:2021](#)

Cette catégorie réfère à des lacunes ou des défauts dans les étapes de conception qui ne prennent pas en compte les principes de sécurité, laissant ainsi le champ libre à diverses vulnérabilités.

Voici de **bonnes pratiques** que vous pourriez mettre en place et des points de vigilance sur lesquels vous pourriez vous focaliser :

1. Stockage non protégé des informations d'identification : la gestion des identités et des accès utilisateurs (mot de passe ou système de génération de certificats) doit être prévue pour être la plus fiable qu'il soit possible d'être.(SHA-256)

2. Violation des limites de la confiance : Il arrive parfois que dans le même "coffre-fort" numérique, on mélange sans le vouloir des objets de valeur avec des objets sans importance. Cette situation se produit lorsqu'on ne distingue pas clairement les données fiables des données non fiables. Sans un mécanisme rigoureux de validation des entrées, le système back-end pourrait devenir la porte dérobée parfaite pour des échanges de données non sécurisées (Vérifier toujours les droits d'un utilisateur avant d'accéder à des informations sensibles)

3. Critères d'observabilité : l'observabilité est caractérisée par la capacité de déduire l'état d'un système ou d'une application par rapport à ce qu'elle renvoie comme information. Il peut s'agir de journaux produits par exemple ou encore de métriques.(mise en place de logs et de système de surveillance en temps réel)

4. Gestion des rôles : l'architecture d'un système, application comme infrastructure, doit être étudiée en prenant en compte les différents rôles qui seront prévus, ou configurables. Une matrice d'habilitation à placer en fonction des rôles peut être un point de départ à la réflexion.

[Secure Product Design - OWASP Cheat Sheet Series](#)

[A05 Security Misconfiguration - OWASP Top 10:2021](#) et [A06 Vulnerable and Outdated Components - OWASP Top 10:2021](#)

Les vulnérabilités résultant d'une **mauvaise configuration** ou de l'**utilisation de composants obsolètes** peuvent offrir aux attaquants une porte ouverte pour compromettre non seulement les applications elles-mêmes mais aussi les données sensibles qu'elles gèrent.

Les formes de mauvaises configuration de sécurité les plus fréquentes :

1. Le durcissement insuffisant des couches applicatives

2. La présence de fonctionnalités inutiles : Des ports ouverts non utilisés, des services superflus, des pages ou des comptes non essentiels, sont autant de points d'entrée potentiels pour des menaces.

3. Traitement des erreurs et fuites d'informations

4. Composants logiciels vulnérables et obsolètes

CVE (Common Vulnerabilities and Exposures) : référentiel international qui recense, identifie et classe les vulnérabilités de sécurité connues dans les logiciels et matériels.

[Guide d'hygiène informatique de L'ANSSI](#) ; [OWASP Dependency-Check](#)

Lorsqu'une CVE (Common Vulnerabilities and Exposures) est publiée concernant une application que vous utilisez, il est important de prendre des **mesures immédiates** pour minimiser les risques de sécurité.

1. **Informez-vous sur la CVE**
2. **Consultez les recommandations du fournisseur**
3. **Appliquez les correctifs**
4. **Mettez en œuvre des mesures d'atténuation**

A07 Identification and Authentication Failures - OWASP Top 10:2021

Un cookie HTTP (également appelé cookie web, cookie Internet, cookie de navigateur ou simplement cookie) est une petite donnée envoyée depuis un site web et stockée sur l'ordinateur de l'utilisateur par un navigateur web lorsque l'utilisateur consulte le site.

Les **cookies de suivi** sont un type de cookie spécifique distribué, partagé et lu sur au moins deux sites web non liés. Le but est de collecter des informations ou éventuellement de vous présenter des données personnalisées.

Recommandations générales

Authentification multifacteur (MFA)

Informations d'identification par défaut

Gestion des mots de passe

Gestion de session

Protection contre les attaques d'énumération de compte

Recommandations liées aux cookies

Attributs Secure et HttpOnly, Attribut SameSite, Durée de vie limitée

Utilisez des Frameworks de Sécurité

- ASP.NET Core Identity Framework pour les applications .NET ;
- Ruby Omniauth pour Ruby ;
- Java JAAS pour Java ;
- PHP PHPSec pour PHP.

A08 Software and Data Integrity Failures - OWASP Top 10:2021

Effectuez systématiquement des vérifications sur les bibliothèques ou les plug-ins avant la mise en place dans le code ou dans l'application

- **Date de la dernière mise à jour** : Il n'est pas impossible qu'un plug-in, ou une bibliothèque, ait son développement abandonné, laissant des vulnérabilités potentiellement présentes.
- **Qui fournit l'élément ? Qui en est l'auteur ?** Est-ce une entreprise reconnue, un indépendant qui au contraire n'est pas connu pour ses contributions ? L'auteur a-t-il une activité régulière autour de son code ?
- **La bibliothèque ou le plug-in est-il nécessaire ?** Il serait dommage d'augmenter le risque de vulnérabilité en installant une bibliothèque ou un plug-in alors que des mécanismes gérant la même fonctionnalité sont déjà présents, dans un framework utilisé ou via un autre composant déjà en place.
- **Le code source est-il disponible ?** Y a-t-il une possibilité d'analyser le code pour être sûr de ne pas créer d'effet de bord ?
- **À quelles ressources le composant a-t-il accès ?** Les permissions demandées sont-elles cohérentes avec la fonction ?

[La CNIL publie un guide RGPD pour les développeuses et développeurs](#)

A09 Security Logging and Monitoring Failures - OWASP Top 10:2021

Un **log**, ou journal, est un enregistrement qui trace les actions, événements ou communications qui se produisent au sein des systèmes informatiques. Les logs peuvent être générés à plusieurs niveaux : applicatifs, réseaux mais aussi directement par le système d'exploitation.

La **journalisation** est le processus d'enregistrement et de stockage de ces logs. Elle joue un rôle important dans le domaine de la cybersécurité et la gestion informatique, car elle fournit une trace historique des opérations et des interactions au sein des systèmes.

La journalisation permet plusieurs choses :

1. **Détection des incidents de sécurité** : Les logs permettent d'identifier des activités suspectes ou malveillantes en enregistrant des tentatives d'accès non autorisées, des modifications inattendues de fichiers ou de configurations et d'autres anomalies.
2. **Audit et conformité** : Les logs fournissent des preuves essentielles pour les audits de conformité réglementaire. Ils prouvent que les contrôles de sécurité sont en place et fonctionnent correctement.
3. **Analyse post-incident** : Après une violation de sécurité, les logs sont souvent l'une des premières sources utilisées pour comprendre ce qui s'est passé, comment l'incident a été déclenché, et quels systèmes ont été affectés.

4. **Amélioration continue** : L'analyse régulière des logs aide à identifier les tendances et les vulnérabilités potentielles, permettant ainsi aux organisations d'améliorer continuellement leurs mesures de sécurité.
5. **Support et maintenance** : Les logs aident à diagnostiquer et résoudre les problèmes techniques, en fournissant un aperçu détaillé des performances du système et des erreurs éventuelles.

[A10 Server Side Request Forgery \(SSRF\) - OWASP Top 10:2021](#)

SSDLC

1. Planification

- Quels sont les objectifs à court et à long terme de l'application ?
- Comment pouvons-nous mesurer le succès du projet ?

[Application Security Verification Standard 4.0.3 \(fr\)](#) : liste de contrôle de sécurité

2. Analyse des besoins

- Quelles sont les ressources à protéger ?
- Quels types d'attaques devons-nous anticiper ?
- Comment pouvons-nous assurer la confidentialité, l'intégrité et la disponibilité des données traitées par l'application ?

3. Design/Conception

- Définition de l'architecture
- Interfaces utilisateur et système
- Gestion des données
- Choix des technologies
- Sécurité : Intégrer les considérations de sécurité dès la conception en prévoyant des mesures telles que le chiffrement des communications, la gestion sécurisée des sessions, et la protection des données utilisateur.

Modéliser les menaces lors de la conception de votre application

Un modèle de menace typique comprend :

- **Description du sujet** : Qu'est-ce qu'on modélise exactement ?
- **Hypothèses** : Quelles sont les suppositions que nous pourrions remettre en question à l'avenir ?
- **Menaces potentielles** : Quels dangers guettent notre système ?
- **Actions de mitigation** : Comment pouvons-nous contrer ces menaces ?
- **Validation et vérification** : Comment s'assurer que notre modèle et nos actions sont efficaces ?

[Threat Modeling Process | OWASP Foundation](#) : créer une modélisation de menace sur une application

[MITRE ATT&CK®](#) (“Adversarial Tactics, Techniques and Common Knowledge”).



les différentes étapes d'une cyberattaque

[Qu'est-ce que la modélisation des menaces ? | Cloudflare](#)

4. Implémentation

Un linter est un outil qui analyse automatiquement le code source pour détecter les erreurs, les bugs potentiels, et les écarts par rapport aux conventions de codage, aidant ainsi à améliorer la qualité et la sécurité du code.

analyse de code statique (SAST - Static Application Security Testing) : examine votre code source sans exécuter le programme, offrant ainsi la possibilité de détecter les failles de sécurité au niveau de l'implémentation avant la compilation et le déploiement. Cette approche, dite de test en **boîte blanche**, analyse directement votre code pour y repérer des **vulnérabilités potentielles**, permettant leur **correction en amont** de la mise en production.

[SonarQube Server](#) : outil SAST

analyse de code dynamique (DAST - Dynamic Application Security Testing) : examine l'application lorsqu'elle est en cours d'exécution, adoptant une **méthode de test en boîte noire** pour identifier les **vulnérabilités** qui se manifestent uniquement durant l'exécution du programme. Cette approche simule des attaques automatisées contre votre application pour découvrir des issues que les attaquants pourraient exploiter, sans nécessiter d'accès au code source interne.

[Zed Attack Proxy \(ZAP\)](#) : outil DAST

[SAMM \(Software Assurance Maturity Model\)](#)

[OWASP Dependency-Check](#) - [OWASP Dependency-Track](#) : surveiller l'utilisation des bibliothèques tierces

[OWASP Cheat Sheet Series](#) : implémenter des fonctionnalités de manière sécurisée

5. Test et Intégration

Validation du fonctionnement de l'application et de la sécurité de bout en bout.

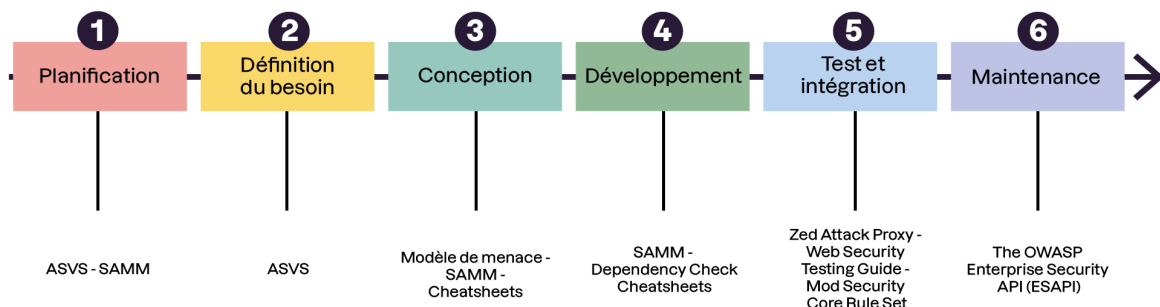
[WSTG - v4.2 | OWASP Foundation](#) : scénarios de tests applicables à votre application.

[GitHub - OWASP/www-project-modsecurity-core-rule-set](#) : détecter et bloquer les attaques contre votre application web lors de l'étape de déploiement en production.

6. Maintenance

Correction des bugs signalés par les utilisateurs et **l'amélioration continue** de l'application.

https://www.owasp.org/index.php/Category:OWASP_Enterprise_Security_API : API pour sécuriser des applications web



Ressources à utiliser en fonction du SDLC

